SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment 4.2

Program	**: B. Tech (CSE)**
Course Title	**: AI Assisted Coding**
Course Code	**: 23CS002PC304**
Semester	**: III**
Academic Session	**: 2025-2026**
Name of Student	**: Sankeerthana**
Enrollment No.	**: 2403A51L41**
Batch No.	**: 52**
Date	**: 20-01-2026**

Task 1

Prompt 1: Write a Python function to determine whether a given number is prime

Code:

```python
def is_prime(n):
    """Check if a number is prime."""
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
# Example usage
number = 29
if is_prime(number):
    print(f"{number} is a prime number.")
else:
    print(f"{number} is not a prime number.")
```

Output:

```
29 is a prime number.
```

Explaination: The function is_prime checks if a number n is prime by first ruling out numbers less than
or equal to 1. It then checks for factors from 2 up to the square root of n. If any factor

factors are found, it returns True, indicating that n is pr divides n evenly, the function returns False, indicating that n is not prime.

TASK2

Prompt : write a python function that calculates the sum of elements in a list.

For example:1 2 3 4 is 10

Code:

```
def sum_list(numbers):
    total = 0
    for num in numbers:
        total += num
    return total

# Example usage
numbers = [1, 2, 3, 4,]
print(f"The sum of {numbers} is {sum_list(numbers)}")
```

Output:

```
The sum of [1, 2, 3, 4] is 10
```

Explaination: The function sum_list takes a list of numbers as input and initializes a variable total to 0. It then iterates through each number in the list, adding it to total. Finally, it returns the total sum of the elements in the list.

TASK 3:

Prompt:create python a function that extracts digits from an alphanumeric string

Example: IO:wsdfxghi876 OP:123,IO:54weshjku6 OP:546,IO:p8765rdchuo7 OP:8765

Code:

```
import re
def extract_digits_from_string(s):
    digits = re.findall(r'\d+', s)
    return ''.join(digits)
# Example usage:
print(extract_digits_from_string("abc123xyz"))  #
print(extract_digits_from_string("54weshjku6"))
print(extract_digits_from_string("p8765rdchuo7"))
```

Output:

```
123
876
7546
```

Explaination:
The function 'extract_digits_from_string' uses regular expressions to find all sequences of digits in the input string 's'.

TASK 4:

Prompt: Generate a function that counts the number of vowels in a string.

CODE: Zero shot:

```
def count_vowels(s):
    vowels = "aeiouAEIOU"
    count = 0
    for char in s:
        if char in vowels:
            count += 1
    return count

# Example usage:
print(count_vowels("Hello World"))
```

OUTPUT:

```
3
```

CODE: Few shot

```
def count_vowels(input_string):
    vowels = "AEIOUaeiou"
    count = 0
    for char in input_string:
        if char in vowels:
            count += 1
    return count

print(count_vowels("AILAB"))
print(count_vowels("LAPTOP"))
print(count_vowels("DESKTOP"))
```

OUTPUT:

```
3
2
2
```

COMPARISION:

In the one-shot example, only a single input string is provided along with the expected output,which may not fully illustrate the task of counting vowels in various contexts.
In the few-shot example, multiple input strings are provided along with the expected output,which helps to clarify the task of counting vowels in different contexts.

TASK5

PROMPT: Generate a function that find the minimum of three numbers without using the python built-in min() function
Example 1:
Input:(3,1,2)
Output:1
Example 2:
Input:(-1,2,1)

Output:-1
Example 3:
Input:(7,7,7)
Output:7

CODE:

```
def find_minimum(a, b, c):
    if a <= b and a <= c:
        return a
    elif b <= a and b <= c:
        return b
    else:
        return c
# Test Cases
print(find_minimum(3, 1, 2))
print(find_minimum(-1, 2, 1))
print(find_minimum(7, 7, 7))
```

OUTPUT:

```
1
-1
7
```

Explaination:

It compares the three numbers using conditional statements to determine which one is the smallest.
It returns the smallest number among the three.
The test cases demonstrate the function's ability to find the minimum value in different scenarios.